Evidence-Gated Evaluation of Agentic AI Systems

A Bounded Framework for Testing Capability Without Expanding Authority

Tom Budd
ResoVerse LLC, San Diego, California, USA
tom@tombudd.com
tombudd.com
26 July 2026

ABSTRACT

Evaluations of agentic artificial intelligence systems increasingly measure whether a system can complete multi-step tasks, use tools, withstand adversarial inputs, or exhibit dangerous capabilities. These measurements are necessary, but they are not equivalent to authorization. A system can demonstrate a capability without being permitted to exercise it, and an evaluation can create new risk when its environment silently grants tools, data, credentials, or side effects beyond the stated research question. This methods paper introduces Evidence-Gated Evaluation of Agentic Systems (EGEAS), a framework that binds six objects before execution: system identity, bounded claims, an authority envelope, a frozen protocol, an evidence ledger, and a decision gate. The framework defines four terminal decisions: PASS, HOLD, FAIL, and INVALID_RUN. A PASS supports only the predeclared claim within the tested environment and produces no authority delta. Ambiguity, incomplete evidence, missing approval, or unfrozen methods cannot be converted into success.

The paper provides a threat model, formal gate semantics, a claim-to-evidence structure, and a minimum adversarial test matrix. An accompanying executable package contains a closed JSON Schema, 18 synthetic valid and mutated cases, deterministic gate logic, and regression tests. The package performs no model calls and creates no external effects. In the included demonstration, all 18 cases produced their frozen expected decisions and five regression tests passed. This result validates the internal consistency of the example gate, not the safety, alignment, or deployment readiness of any AI system. EGEAS is offered as a falsifiable methods proposal for evaluation teams that need capability evidence without treating evaluation success as permission to act.

Keywords: AI evaluation, agentic systems, alignment evaluation, evidence governance, authority boundaries, adversarial testing, reproducibility, human oversight

1. Introduction

AI evaluations are becoming part of decisions about model development, release, deployment, and risk controls. Existing work has distinguished dangerous-capability evaluations from evaluations of whether a system is likely to apply capabilities harmfully [1]. Broader evaluation programs have emphasized standardized scenarios, multiple metrics, transparent prompts and outputs, and reproducible tooling [2]. Agent-specific benchmarks extend evaluation into interactive environments where systems plan, call tools, receive observations, and act over multiple turns [3]. More recent work relates agent success to the length of tasks completed by human professionals, providing a view of capability that is closer to real work than static question answering [4].

These developments sharpen an operational problem. Capability and authority are different variables.

An agent may be able to draft an email without being authorized to send it. It may classify a transaction without being authorized to move money. It may identify a software change without being authorized to modify a repository. It may pass a synthetic test without being ready for a live environment. Yet evaluation artifacts often emphasize what a system did and leave under-specified what the evaluation itself was allowed to access, what side effects were possible, what evidence was missing, and what decision follows from the result.

This creates three failure paths.

First, evaluation execution can widen the system's effective authority. A test harness may provide credentials, network access, tools, persistent memory, or external targets that were not required by the research question. The evaluator then measures performance inside a more permissive environment than intended.

Second, evaluation interpretation can widen claims. A result obtained from a narrow fixture set may be summarized as general capability, safety, reliability, or readiness. The evidence may support a local statement while the report implies a broader one.

Third, evaluation success can be mistaken for deployment permission. A PASS may be operationally treated as approval even when no person or policy actually authorized the next action.

The NIST Generative AI Profile treats governance, content provenance, pre-deployment testing, and incident disclosure as linked risk-management considerations [5]. W3C PROV provides a general model for representing entities, activities, agents, and their provenance relationships [6]. The UK AI Security Institute's Inspect framework separates datasets, solvers, and scorers, and supports agent evaluations within sandboxed environments [7]. These are important components. EGEAS addresses a narrower integration question: how should an evaluation bind claims, evidence, and execution scope so that the result cannot silently become a broader permission?

The proposed answer is a precommitted evaluation contract and a zero-authority gate. Before execution, the evaluator freezes:

1. the exact system and artifact under test;
2. the exact claim that the evidence may support;
3. the resources and operations the evaluation may use;
4. the protocol, fixtures, scorers, and thresholds;
5. the evidence and exception records that must be retained; and
6. the decision rule, including the meaning of uncertainty.

The framework's core invariant is:

> *Capability evidence can support a bounded claim. It cannot grant authority.*

This paper makes four contributions:

7. A six-object contract for evidence-gated agent evaluation.
8. Formal PASS, HOLD, FAIL, and INVALID_RUN semantics that preserve uncertainty.
9. A threat model and adversarial matrix covering scope widening, provenance loss, approval substitution, and claim inflation.
10. A public synthetic package demonstrating deterministic gate behavior without a live model, tools, credentials, or external effects.

EGEAS is not a proof that a system is aligned. It is not a replacement for red teaming, security engineering, sociotechnical analysis, field validation, or governance. It is a bounded method for making the relationship among test scope, evidence, claims, and authority explicit.

2. Related Work and the Remaining Gap

2.1 Capability and alignment evaluation

Shevlane et al. describe model evaluations as inputs to risk assessment and governance, distinguishing dangerous capabilities from a model's propensity to apply them harmfully [1]. This distinction helps prevent a single performance measure from carrying the full burden of a safety conclusion. EGEAS adopts the same caution and adds a second separation: neither kind of evaluation grants operational authority.

HELM treats evaluation as a multi-scenario, multi-metric problem and makes raw prompts and completions available for analysis [2]. This supports comparison and transparency. EGEAS borrows the principle that evaluation should expose tradeoffs and missing coverage rather than collapse everything into one score.

AgentBench evaluates language models as agents across interactive environments and documents common failures in long-horizon reasoning, decision-making, and instruction following [3]. METR's task-horizon work emphasizes success probability, human task duration, uncertainty, and external-validity limitations [4]. Together, these lines of work show why agent evaluation cannot be reduced to a static output check. Trajectories, environments, tools, and success definitions matter.

2.2 Evaluation assumptions and benchmark integrity

Barnett and Thiergart argue that evaluation-based safety decisions depend on assumptions about threat coverage, proxy-task validity, and capability elicitation, and that these assumptions should be declared and justified [8]. EGEAS operationalizes this idea through a frozen claim contract, stated exclusions, and a HOLD decision for inadequately justified assumptions.

Benchmark contamination and adaptive overfitting complicate the interpretation of evaluation results. VarBench uses dynamic variable perturbation to create fresh test cases and reduce leakage concerns [9]. EGEAS does not prescribe one contamination defense, but it requires fixture identity, digests, generation methods, and contamination limitations to be recorded as evidence.

2.3 Agent security and authorization

Agent Security Bench formalizes attacks across system prompts, user inputs, memory, planning, and tool use, illustrating that an agent's operational path creates multiple attack surfaces [10]. EGEAS uses this observation to require negative tests at both the bundle boundary and the trajectory boundary. A successful final answer does not erase an unauthorized intermediate action.

South et al. distinguish authentication, authorization, and auditability for delegated AI agents and propose explicit, restricted permission scopes [11]. EGEAS is complementary. Authenticated delegation governs what an agent is permitted to do in an operational environment. EGEAS governs what an evaluator is permitted to expose during a test and what claims may be inferred afterward.

Traditional security practice also supplies a relevant principle: least privilege. The evaluation environment should expose only the resources and operations required to answer the frozen research question. A capability benchmark does not need to become a general-purpose execution environment.

2.4 Provenance and reproducibility

W3C PROV supplies a useful conceptual foundation for linking evidence objects to the activities and agents that generated or used them [6]. EGEAS uses a simplified provenance ledger rather than requiring a specific ontology. Every supported claim must be traceable to protocol identity, fixture identity, raw results, scorer output, environment metadata, exceptions, and review state.

Inspect expresses an evaluation as a dataset, solver, and scorer and can place tool use inside a sandbox [7]. EGEAS adds a surrounding decision contract. The dataset-solver-scorer unit is necessary but does not by itself define who may approve wider use, how incomplete evidence is represented, or whether a passing score changes system authority.

2.5 Gap statement

Existing evaluation frameworks primarily ask whether a system performs, fails, or exhibits risk under specified conditions. Authorization frameworks ask what an authenticated system may do. Provenance frameworks ask where evidence came from. EGEAS connects these concerns at the evaluation boundary:

> *What evidence may a test produce, what claim may that evidence support, and what authority remains unchanged regardless of the score?*

3. Design Requirements

EGEAS is designed around eight requirements.

R1. Exact subject binding

The evaluation must identify the exact system, version, configuration, model dependency, tool configuration, and artifact digest. A changed artifact is a new subject unless the protocol explicitly defines the change as an experimental variable.

R2. Closed claim surface

Every evaluated claim must include an operational statement, metric, threshold, population or scenario boundary, and exclusions. Open language such as "safe," "aligned," "autonomous," or "production ready" is inadmissible without an operational definition and evidence that covers the implied scope.

R3. Least-authority evaluation

The evaluation environment must enumerate resources, operations, tools, data classes, network status, credential status, persistence, time bounds, and permitted side effects. Wildcards and implied permissions are rejected.

R4. Pre-execution freeze

Research questions, fixtures, scorers, thresholds, stop conditions, and decision rules must be frozen before results are observed. Changes require a new protocol version and must remain visible in the provenance record.

R5. Negative evidence retention

Failures, refusals, malformed inputs, timeouts, missing observations, contradictory evidence, and evaluator errors are evidence. They cannot be deleted merely because they complicate the narrative.

R6. Reproducible evidence chain

The evaluation must retain sufficient information for a qualified reviewer to reconstruct what was tested, with what inputs, under what environment, using what scorer, and with what exceptions.

R7. Fail-closed ambiguity

Ambiguity does not become success. Missing evidence, missing approval, unresolved contradictions, or an incomplete protocol produce HOLD or INVALID_RUN. A demonstrated boundary violation produces FAIL.

R8. Zero authority delta

The decision function returns an evidentiary status only. No terminal result grants tools, credentials, deployment, publication, evidence admission, or wider execution scope. Any subsequent permission requires a distinct authorization process with a named human or institutional authority.

4. The Six-Object EGEAS Contract

Let an evaluation bundle be:

$$B = (S, C, A, P, E, G)$$

where:

- S is the system identity contract;
- C is the bounded claim set;
- A is the evaluation authority envelope;
- P is the frozen protocol;
- E is the evidence ledger; and
- G is the decision gate.

4.1 System identity contract S

The minimum identity contract includes:

- logical system identifier;
- immutable version identifier;
- artifact digest;
- model and model-version identifiers, when applicable;
- agent framework and version;
- prompt, policy, memory, and tool-configuration digests;
- evaluator-controlled environment identifier; and
- declared owner or accountable role.

Identity must be consistent across the protocol, results, and report. If a model provider silently changes a model behind a stable alias, the evaluator must treat the run as version-uncertain and limit the claim accordingly.

4.2 Bounded claim set C

Each claim c_i is:

$$c_i = (q_i, m_i, t_i, d_i, x_i)$$

where q_i is the operational statement, m_i the metric, t_i the threshold, d_i the tested domain, and x_i the exclusions.

Example:

> *On synthetic ticket fixtures v1, system alpha classifies the declared routing category with exact-match accuracy of at least 0.90. This claim excludes live tickets, external communication, account changes, and production reliability.*

This is intentionally narrower than "system alpha can autonomously triage IT tickets."

4.3 Authority envelope A

The authority envelope is a closed set:

$$A = (R, O, D, T, N, K, X, H)$$

where:

- R: exact resources;
- O: exact operations;
- D: permitted data classes;
- T: time and volume limits;
- N: network policy;
- K: credential policy;
- X: permitted external effects; and
- H: required human approvals.

The envelope governs the evaluation harness, not just the model. If the model is denied network access but the harness sends its output to an external service, the evaluation has an external effect and must declare it.

The closed-world rule is:

$$operation \notin O \text{ or } resource \notin R \text{ implies DENY}$$

An authority envelope must never be inferred from the model's capability profile. The ability to call a tool does not place that tool in O.

4.4 Frozen protocol P

The protocol includes:

- research question and hypotheses;
- threat model;
- fixture-generation and selection method;
- contamination controls;
- baseline and comparison conditions;
- model elicitation and sampling settings;
- tools and sandbox configuration;
- scorer definitions and scorer digests;
- acceptance thresholds;
- negative and adversarial cases;
- run count and statistical plan;
- stop conditions;
- exception-handling rules; and
- a versioned protocol digest.

Protocol changes after observing results are allowed only as a new study iteration. The earlier result and rationale for revision remain in the ledger.

4.5 Evidence ledger E

The ledger records:

- input and fixture digests;
- raw outputs and trajectories;
- tool requests and tool results;
- scorer outputs;
- environment and dependency manifest;
- timestamps or deterministic fixture time;
- random seeds and sampling parameters;
- evaluator identity and role;
- approvals, when required;
- exceptions, timeouts, and missing data;

- protocol deviations;
- result digests; and
- independent-review status.

The ledger distinguishes three evidence states:

11. **Recorded:** an artifact exists and is linked.
12. **Validated:** integrity and schema checks pass.
13. **Admitted:** a named decision-maker or independent process accepts the evidence for a specific claim.

Self-generated evidence cannot silently admit itself.

4.6 Decision gate G

The gate returns one of four values:

- **PASS:** all declared obligations are satisfied, thresholds are met, and no blocking exception or boundary violation exists.
- **HOLD:** the run is well-formed, but required evidence, approval, coverage, or exception resolution is incomplete.
- **FAIL:** evidence demonstrates that a declared acceptance criterion or authority boundary was violated.
- **INVALID_RUN:** the bundle is malformed, internally inconsistent, unfrozen, or not bound to the claimed subject.

Formally:

G(B) = INVALID_RUN if the contract is invalid; FAIL if evidence shows a boundary or blocking-criterion violation; HOLD if evidence is incomplete, contradictory, or approval-dependent; otherwise PASS when all declared criteria are met.

For every result r:

$$\Delta Authority(r) = 0$$

The gate can recommend a later decision. It cannot execute that decision.

5. Threat Model

EGEAS assumes an evaluator controls the protocol and test harness but may make mistakes, use incomplete evidence, or face adversarial system behavior. The system under test may be stochastic, may exploit available tools, and may respond differently when it detects evaluation. External services and model providers may change. Independent reviewers may be unavailable.

The framework addresses eight threat classes.

T1. Subject substitution

The tested artifact differs from the reported artifact. Controls include exact digests, immutable versions, and cross-object identity checks.

T2. Scope leakage

The environment exposes undeclared resources or operations. Controls include closed allowlists, static inspection, runtime denial tests, and a complete harness-level authority envelope.

T3. Approval substitution

Model confidence, prior success, evaluator convenience, or an automated score substitutes for required human approval. The gate returns HOLD when approval evidence is absent.

T4. Provenance loss

Raw outputs, trajectories, failures, or environment details are missing. The gate returns HOLD or INVALID_RUN rather than reconstructing favorable evidence through inference.

T5. Claim inflation

A local result is summarized as a broader capability, safety, or readiness claim. Controls include closed claims, exclusions, a claim-to-evidence ledger, and an explicit prohibited-claims section.

T6. Scorer ambiguity

Scorer rules are unstable, subjective, changed after results, or dependent on an unpinned model grader. Controls include scorer versioning, calibration cases, inter-rater checks where appropriate, and an unfrozen-protocol failure.

T7. Negative-result suppression

Failed attempts, unsafe trajectories, or null results are discarded. Controls include complete run accounting and evidence-manifest reconciliation.

T8. Gate-to-action confusion

A PASS is treated as authorization to deploy or widen access. Controls include the zero-authority invariant, separate decision artifacts, and explicit labeling in every report and machine-readable result.

EGEAS does not solve evaluator collusion, compromised hardware, malicious data providers, undisclosed model updates, or every form of sand-box escape. These risks must be declared as limitations or handled by additional controls.

6. Minimum Evaluation Procedure

Step 1. Freeze the research question

Write a question that can be falsified. Replace "Is the agent safe?" with a bounded question such as:

> *Under fixture set F and authority envelope A, does the agent request any operation outside O during task family Q?*

Step 2. Bind the exact subject

Record system, model, prompt, policy, memory, tool, and environment versions. Produce digests for immutable artifacts.

Step 3. Declare the authority envelope

List every resource and operation. State whether network, credentials, personal data, persistence, external effects, subprocesses, and human approvals exist. Reject wildcards.

Step 4. Build a claim-evidence matrix

For each prospective claim, record:

- the required evidence;
- the metric and threshold;
- exclusions;
- known confounders;
- negative controls; and
- the strongest permitted wording if the test passes.

Step 5. Freeze protocol and adversarial cases

Predeclare fixtures, mutations, scoring, stop criteria, and expected failure behavior. Include malformed bundles, unknown operations, identity mismatches, missing evidence, and attempted authority widening.

Step 6. Execute under containment

Enforce the declared envelope at the harness boundary. Record all attempts, including denied requests and failed runs. Stop if the actual environment differs materially from the frozen environment.

Step 7. Reconcile the evidence ledger

Compare expected run count with raw results. Verify digests, environment metadata, scorer outputs, exceptions, and approvals. Do not silently drop ambiguous rows or failed cases.

Step 8. Apply the frozen gate

Compute PASS, HOLD, FAIL, or INVALID_RUN. Preserve the reason codes and evidence references.

Step 9. Calibrate the claim

Publish the narrowest statement supported by the evidence. Include limitations and prohibited inferences.

Step 10. Seek independent review

Ask a reviewer who did not author the protocol or implementation to reproduce the run, inspect the threat model, challenge the scorer, and test the claim boundary. Until review is complete, label the artifact accordingly.

7. Minimum Adversarial Matrix

Table 1 defines a minimum matrix. It is not comprehensive.

Class	Mutation	Expected gate	Reason
Identity	Artifact digest mismatch	INVALID_RUN	Wrong or ambiguous subject
Schema	Unknown field or missing required field	INVALID_RUN	Contract not closed
Protocol	Scorer missing or protocol unfrozen	INVALID_RUN	Result not precommitted
Authority	Wildcard resource	FAIL	Silent scope widening
Authority	Unknown operation	FAIL	Operation outside closed set
Authority	Network enabled when prohibited	FAIL	Environment violates envelope
Authority	Credentials present when prohibited	FAIL	Evaluation creates access risk
Data	Real-person data introduced	FAIL	Data boundary violated
Effects	External action becomes possible	FAIL	Side-effect boundary violated
Evidence	Raw results missing	HOLD	Claim cannot be reconstructed
Evidence	Environment manifest missing	HOLD	Reproduction boundary unknown
Exception	Timeout blocks required cases	HOLD	Incomplete coverage
Approval	Required approval absent	HOLD	Human decision cannot be inferred
Claim	Exclusions omitted	INVALID_RUN	Claim surface is not bounded
Review	Independent review pending	PASS with limitation	Internal result may stand; external validation is not claimed

The distinction between FAIL and INVALID_RUN is useful. FAIL means a valid test produced adverse evidence or crossed a boundary. INVALID_RUN means the evaluation cannot support a conclusion because its contract was malformed or unfrozen. HOLD preserves a path to resolution without converting missing evidence into failure or success.

8. Synthetic Demonstration

8.1 Purpose

The demonstration tests only whether the example machine-readable gate implements the semantics described in this paper. It does not evaluate an AI model or agent.

8.2 Materials

The reproducibility package contains:

- a draft 2020-12 JSON Schema for a minimal bundle;
- one complete base fixture;
- 17 derived mutations, for 18 cases total;
- deterministic canonical JSON and SHA-256 result digests;
- a validator implementing the four gate decisions;
- five regression tests; and

- a generated result ledger and file manifest.

The validator does not call a model, network, tool, credential, database, subprocess, or external service. It reads local JSON fixtures and writes a local result file.

8.3 Frozen expectations

The 18 cases were assigned expected decisions before execution:

- 2 PASS;
- 4 HOLD;
- 6 FAIL; and
- 6 INVALID_RUN.

The PASS cases include one complete bundle and one bundle with independent review marked PENDING. The latter is intentionally a PASS only for internal synthetic consistency. It cannot be described as independently validated.

8.4 Results

All 18 cases matched their frozen expected gate. Five regression tests passed. Repeated canonicalization of equivalent JSON objects produced identical bytes and digests.

These results demonstrate:

14. the example gate distinguishes malformed evaluation contracts from boundary violations;
15. missing evidence and approval dependencies remain HOLD rather than becoming PASS;
16. the example PASS state does not require a false claim of independent review; and
17. the machine-readable result declares `authority_delta: NONE` and `external_effects: false`.

These results do not demonstrate:

- model capability;
- resistance to prompt injection;
- secure sandboxing;
- real-world alignment;
- production reliability;
- effectiveness across organizations;
- legal or regulatory compliance; or
- superiority to another evaluation framework.

8.5 Reproduction

Using Python 3.11 or later with no third-party dependencies:

```
python validate_bundle.py
python -m unittest -q
```

The package includes exact expected outputs and a SHA-256 manifest. A clean independent reproduction remains pending.

9. Reporting Template

Every EGEAS report should begin with five statements:

18. **Subject:** exact system, version, and digest.
19. **Claim:** exact supported statement and exclusions.
20. **Authority:** resources and operations allowed during evaluation.
21. **Decision:** PASS, HOLD, FAIL, or INVALID_RUN with reason codes.
22. **Authority delta:** NONE.

A concise result should read:

> *PASS for CLM-001 on protocol PRO-SYNTHETIC_V1 and fixture set F1. The evidence supports only the stated synthetic classification claim. Independent review is pending. This result does not authorize live data, network access, credentials, external delivery, deployment, or expanded authority.*

The report should also list:

- all failed and incomplete cases;
- protocol deviations;
- missing evidence;
- evaluator and reviewer roles;
- limitations;
- prohibited claims; and
- the exact next decision that requires separate authorization.

10. Discussion

10.1 Why zero authority delta matters

Many governance documents say that humans remain in control, but the operational transition from test result to action is often implicit. A score appears in a dashboard, a threshold is met, and a downstream system changes state. EGEAS treats that transition as a separate object.

This separation prevents two opposite errors. A successful capability evaluation does not automatically permit use. An adverse result does not automatically authorize remediation actions that were outside the evaluator's scope. Both deployment and repair can create effects and therefore need independent authority.

10.2 PASS is intentionally narrow

PASS is not the strongest possible word. It is the most bounded terminal state. It means that the declared obligations were satisfied under the declared conditions. The claim remains conditional on the validity of the fixtures, scorer, elicitation, threat model, and environment.

The framework therefore discourages a universal safety score. Agentic systems operate across heterogeneous tools, data, environments, and human relationships. A single scalar tends to hide the exact conditions that make an action safe or unsafe.

10.3 HOLD is evidence-preserving

HOLD is not indecision. It is a positive representation of unresolved evidence. It separates:

- a system that failed a valid test;
- a test that could not complete;
- a result that lacks required approval; and
- a malformed protocol that cannot support a conclusion.

This distinction is important for research integrity. If incomplete runs are silently excluded, results can become biased toward successful completion. If uncertainty is forced into FAIL, teams may overstate what the evidence shows.

10.4 Independent review

Independent review is valuable because protocol authors are poorly positioned to detect every assumption they embedded. A reviewer should:

- reproduce from a clean environment;
- verify artifact and fixture digests;
- inspect whether the authority envelope was actually enforced;
- challenge proxy-task validity and capability elicitation;
- inspect negative and null results;
- assess scorer reliability;
- attempt undeclared operations;
- verify that claims do not exceed evidence; and
- confirm that PASS creates no automatic authority.

Independence should be described, not presumed. Employment, funding, authorship, prior implementation work, and other conflicts should be disclosed.

10.5 Relationship to safety cases

EGEAS can supply structured evidence to a safety case but is not itself a safety case. A safety case connects claims, arguments, evidence, context, and rebuttals across a broader system. EGEAS focuses on the integrity of individual evaluation packets and their transition into bounded claims.

10.6 Applicability beyond frontier models

The method applies to enterprise agents, workflow automation, robotics simulations, agent admission systems, and local research prototypes. The stricter the consequence of a tool call, the more important it becomes to distinguish capability from permission.

11. Limitations and Threats to Validity

First, the framework is a methods proposal authored and demonstrated by one researcher. It has not yet received independent technical review.

Second, the synthetic package tests gate semantics, not a live agent. The 18 cases are transparent examples and cannot establish coverage of real-world agent failure modes.

Third, the schema is intentionally minimal. Production use would require richer identity, cryptographic approval, revocation, timing, incident, privacy, and statistical objects.

Fourth, fail-closed policy can reduce usefulness or produce excessive HOLD decisions when environments are poorly specified. This is a design tradeoff, not a free safety improvement.

Fifth, provenance integrity depends on trustworthy collection. Digests can show that retained bytes did not change, but they do not prove that the original collection was complete or honest.

Sixth, evaluator independence and reviewer competence are sociotechnical conditions. A machine-readable field cannot guarantee either.

Seventh, capability elicitation remains difficult. A FAIL to demonstrate capability may reflect inadequate prompting, scaffolding, tools, time, or sampling rather than true absence [8]. EGEAS requires that limitation to be visible but does not solve it.

Eighth, the framework does not prevent a separate actor from ignoring its result and deploying a system. It improves traceability and decision discipline; it is not an enforcement mechanism unless integrated with one.

Finally, the terminology of authority can be interpreted legally, technically, or organizationally. In this paper, authority means explicit permission to access a resource or perform an operation. It is not a legal conclusion about agency, liability, or fiduciary duty.

12. Research Agenda

Five next studies would materially test the framework.

23. **Independent reproduction.** Can an unaffiliated reviewer reproduce all synthetic decisions from a clean checkout and identify hidden assumptions?
24. **Comparative protocol study.** Does adding EGEAS to an existing agent benchmark improve reviewers' ability to identify unsupported claims and undeclared execution surfaces?
25. **Inter-rater study.** Do evaluators agree on PASS, HOLD, FAIL, and INVALID_RUN when presented with ambiguous evidence packets?
26. **Live but contained agent study.** In a sandboxed, synthetic environment, can adversarial agents induce scope widening or approval substitution despite the declared envelope?
27. **Usability and cost study.** What documentation and execution overhead does the method introduce, and when is that overhead justified by consequence?

The framework should be revised if these studies show that its objects are redundant, its decisions are unreliable, its overhead is disproportionate, or its claim boundaries do not improve interpretation.

13. Conclusion

Agentic AI evaluation needs to answer more than whether a system completed a task. It must also make visible which system was tested, which claim was at stake, what the environment permitted, what evidence was retained, what remained uncertain, and what decision follows.

EGEAS binds those questions into a six-object contract. It treats capability and authority as separate variables, preserves negative and incomplete evidence, and makes PASS a bounded evidentiary status rather than permission to act. The accompanying synthetic package shows that these semantics can be expressed in a small, deterministic, machine-readable gate.

The framework's strongest claim is deliberately modest: evaluation evidence should remain attached to the conditions that produced it. When those conditions are incomplete, the result should remain incomplete. When a boundary is crossed, the result should fail. When a test passes, authority should remain exactly where it was.

Data and Code Availability

The release candidate includes the manuscript source, a JSON Schema, 18 synthetic cases, a deterministic Python validator, regression tests, generated results, and SHA-256 manifests. No private or personal research data are used. Public materials: https://github.com/tombudd/evidence-gated-agent-evaluation. DOI: not assigned.

Ethics and Safety Statement

This work uses synthetic metadata and does not evaluate or deploy a live AI system. It uses no credentials, personal data, external tools, or consequential actions. The framework is intended to reduce claim and authority overreach. It should not be represented as proof that any system is safe, aligned, secure, compliant, or ready for deployment.

Declaration of Interests

Tom Budd is the founder and chief executive of ResoVerse LLC. The author designed the framework and the synthetic demonstration. No independent review had been completed as of 26 July 2026.

Acknowledgments

The author thanks the developers and researchers who publish open evaluation frameworks, benchmarks, standards, and reproducibility materials. Independent technical review of this manuscript is invited.

References

1. Shevlane, T., Farquhar, S., Garfinkel, B., et al. (2023). Model evaluation for extreme risks. *arXiv:2305.15324*. https://doi.org/10.48550/arXiv.2305.15324
2. Liang, P., Bommasani, R., Lee, T., et al. (2022). Holistic evaluation of language models. *arXiv:2211.09110*. https://doi.org/10.48550/arXiv.2211.09110
3. Liu, X., Yu, H., Zhang, H., et al. (2023). AgentBench: Evaluating LLMs as agents. *arXiv:2308.03688*. https://doi.org/10.48550/arXiv.2308.03688
4. Kwa, T., West, B., Becker, J., et al. (2026). Measuring AI ability to complete long software tasks. *NeurIPS 2025; arXiv:2503.14499v4*. https://doi.org/10.48550/arXiv.2503.14499
5. Autio, C., Dunietz, J., Hall, P., et al. (2024). *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*. NIST AI 600-1. https://doi.org/10.6028/NIST.AI.600-1
6. Lebo, T., Sahoo, S., McGuinness, D., et al. (2013). *PROV-O: The PROV Ontology*. W3C Recommendation. https://www.w3.org/TR/prov-o/
7. UK AI Security Institute. (2026). *Inspect: A framework for large language model evaluations*. https://inspect.aisi.org.uk/
8. Barnett, P., & Thiergart, L. (2024). Declare and justify: Explicit assumptions in AI evaluations are necessary for effective regulation. *arXiv:2411.12820*. https://doi.org/10.48550/arXiv.2411.12820
9. Qian, K., Wan, S., Tang, C., et al. (2024). VarBench: Robust language model benchmarking through dynamic variable perturbation. *arXiv:2406.17681*. https://doi.org/10.48550/arXiv.2406.17681
10. Zhang, H., Huang, J., Mei, K., et al. (2024). Agent Security Bench: Formalizing and benchmarking attacks and defenses in LLM-based agents. *arXiv:2410.02644*. https://doi.org/10.48550/arXiv.2410.02644
11. South, T., Marro, S., Hardjono, T., et al. (2025). Authenticated delegation and authorized AI agents. *arXiv:2501.09674*. https://doi.org/10.48550/arXiv.2501.09674
12. OWASP Foundation. (2025). *Agentic AI: Threats and mitigations*. https://genai.owasp.org/resource/agentic-ai-threats-and-mitigations/
13. National Institute of Standards and Technology. (2020). *Security and Privacy Controls for Information Systems and Organizations*. NIST SP 800-53 Rev. 5. https://doi.org/10.6028/NIST.SP.800-53r5

14. Raji, I. D., Smart, A., White, R. N., et al. (2020). Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing. *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, 33-44. https://doi.org/10.1145/3351095.3372873